Implicit Threading: Fork-Join Merge Sort & OpenMP Matrix Multiplication

Cheolwon Park (Student ID: 2024270616)
Korea University
scott9020@korea.ac.kr

1 Introduction

Implicit threading abstracts away thread management so the programmer can focus on *what* can run in parallel rather than *how* threads are created and scheduled. This report accompanies a Week 5 assignment exercising two such models from Silberschatz 4.5: the **fork-join** pattern with POSIX threads for parallel merge sort (`mergesort.c`), and **OpenMP** for dense matrix multiplication (`matmul.c`). Beyond the required implementation, this archival version adds a depth-threshold sweep, a six-loop-order matmul benchmark, and brief notes on the roofline and cache-blocking directions for further optimization.

2 Background

2.1 Fork-Join Parallelism

Fork-join is a structural pattern in which a task is recursively split (*forked*) into independent subtasks that run in parallel, and whose results are merged (*joined*) once all subtasks finish. It maps naturally onto divide-and-conquer algorithms such as merge sort, quicksort, and tree traversal. The key design question is *when to stop forking*: a naive implementation that spawns a fresh thread at every recursion level creates an exponential number of short-lived threads whose creation cost dominates the actual work, so practical implementations cap parallelism with a depth threshold or task granularity limit.

2.2 OpenMP and Implicit Data Parallelism

OpenMP (Silberschatz 4.5.3) adds parallelism through `#pragma` annotations on loops and code regions. A single directive, `#pragma omp parallel for`, partitions a `for` loop's iteration space across a pool of worker threads managed automatically by the runtime. It fits regular data parallelism where each iteration operates on independent data, and contrasts with fork-join in two ways: parallelism is divided by iteration index rather than recursive subtask, and thread management is fully implicit, with no `pthread_create`, no join, and no argument packing. The programmer specifies *what* can run in parallel; the runtime decides *how*.

2.3 Amdahl's Law and the Limits of Parallelism

Amdahl's Law sets the upper bound on parallel speedup. Let $T(1)$ be the single-threaded wall-clock time, decomposed into a serial part T_s (code that cannot be parallelized) and a parallel part T_p, so $T(1) = T_s + T_p$. Define the serial fraction $f = T_s / T(1)$, giving $T_s = f{\cdot}T(1)$ and $T_p = (1 - f){\cdot}T(1)$.

With N threads and a perfectly scalable parallel part, $T(N) = f{\cdot}T(1) + (1 - f){\cdot}T(1) / N$. The speedup is $S(N) = 1 / (f + (1 - f)/N)$. As $N \to \infty$, $S(N) \to 1/f$: the serial fraction sets a hard upper bound regardless of core count. Small serial fractions matter enormously; $f = 5\%$ caps speedup at 20×, and $f = 1\%$ at 100×.

Amdahl models *strong scaling* (fixed problem size, more threads). Gustafson's Law offers the complementary *weak scaling* view, in which the problem grows with core count and the serial

fraction shrinks relative to the growing parallel work. This assignment uses fixed input sizes (10 million integers, 1024 × 1024 matrix), so Amdahl is the appropriate lens.

2.4 Memory Hierarchy and Spatial Locality

Modern CPUs bridge fast cores and slow DRAM through a stack of caches: L1 (tens of KB, 1–4 cycle latency), L2 (a few MB, ~10 cycles), optional L3 (tens of MB, ~30 cycles), and DRAM itself (100–300 cycles). Data move through the hierarchy in fixed-size *cache lines* (64 bytes on x86, 128 bytes on Apple Silicon P-cores).

Two forms of locality govern cache efficiency. **Spatial locality**: if byte X is accessed, its neighbours are likely accessed soon, so stride-1 sequential reads let one cache-line fill serve many consecutive accesses. **Temporal locality**: recently-used data are likely to be reused, so keeping a working set smaller than L1 means repeated accesses hit cache rather than returning to DRAM. Row-major matrix storage (C's default) yields stride-1 access only when the innermost loop iterates over the last (column) index; any other order pays a stride-N penalty on at least one matrix, which is the mechanism behind the ~14× loop-reorder speedup measured in Part 5.4.

3 Implementation

3.1 Fork-Join Merge Sort

The parallel merge sort is in `mergesort.c`. The driver `merge_sort_parallel` takes a subrange plus a depth counter. If *depth* ≥ `max_depth`, it falls back to the sequential merge sort. Otherwise it forks: `pthread_create` spawns a thread to sort the left half at *depth* + 1, while the current thread recursively sorts the right half. After the right-half recursion returns, it calls `pthread_join` on the left-half thread and merges the two sorted halves.

Why fork only one thread per level? A naive implementation forks both halves, leaving the caller idle inside `pthread_join`. That wastes a CPU core for the duration of the subtree and doubles the total new-thread count across the whole run, from $2^d - 1$ to $2^{d+1} - 2$. Forking only the left half and processing the right half in the caller keeps every live thread busy.

Why a depth threshold? With depth d, up to 2^d threads run concurrently at the leaves of the recursion tree. Too low under-utilizes the cores; too high incurs massive `pthread_create` overhead (tens of microseconds per call) and reserves one stack per thread (typically 512 KB on macOS, 8 MB on Linux). The default is `max_depth` = 3 (up to 8 concurrent threads); the empirical sweet spot is measured in Part 5.2 and discussed in Part 6.2.

3.2 OpenMP Matrix Multiplication

`matmul.c` implements three variants on 1024 × 1024 dense matrices of doubles: a sequential i-j-k triple loop, an OpenMP basic version that adds `#pragma omp parallel for` above the outermost i-loop, and an OpenMP optimized version that reorders the loops to i-k-j and applies the same pragma. In i-k-j, the innermost j-loop walks the column indices of both B and C with stride 1, exploiting spatial locality on two of the three matrices simultaneously. The six possible loop permutations are compared in Part 5.4 and analyzed in Part 6.3.

4 Experimental Setup

4.1 Hardware Environment

All measurements were taken on an Apple M2 Pro: 12 cores arranged as 8 Performance cores and

4 Efficiency cores with no simultaneous multithreading, L1d 128 KB per P-core, L2 16 MB shared across four P-cores, and 16 GB unified LPDDR5 memory at about 200 GB/s of bandwidth. Apple Silicon uses **128-byte** cache lines on P-cores, twice the 64-byte Intel/x86 convention, so each line fill serves 16 consecutive doubles in a stride-1 loop rather than 8. Full specs appear in the table below.

Component	Value
CPU	Apple M2 Pro (Mac14,10)
Cores	12 total: 8 Performance + 4 Efficiency (no SMT)
P-core clock	~3.5 GHz (base), up to ~3.7 GHz boost
L1 instruction (P-core)	192 KB per core
L1 data (P-core)	128 KB per core
L2 (P-cluster, shared)	16 MB (4 P-cores per cluster)
L1d (E-core)	64 KB per core
L2 (E-cluster, shared)	4 MB
Cache line size	128 bytes on P-cores (16 doubles)
Unified memory	16 GB LPDDR5, ~200 GB/s bandwidth
Kernel	Darwin 25.4.0 (macOS), arm64
C compiler (mergesort)	Apple clang 21.0.0 → arm64 native
C compiler (matmul)	Homebrew GCC 15.2.0 (libgomp) → x86_64 via Rosetta 2
Compile flags	-Wall -O2 (+ -pthread for mergesort, -fopenmp for matmul)

4.2 Software Toolchain and Measurement Protocol

`mergesort.c` was built as a native arm64 binary with Apple clang 21 (`-pthread -O2`); `matmul.c` was built with Homebrew GCC 15 and `libgomp` as x86_64 (`-fopenmp -O2`) and runs under Rosetta 2, which adds roughly 15–25% overhead; absolute GFLOP/s in Part 6.4 are therefore conservative, but relative comparisons between matmul variants are unaffected because all run under the same translation layer. OpenMP used the default `OMP_NUM_THREADS` = 12. Each configuration was run three times and the median is reported.

5 Results

5.1 Merge Sort: Sequential versus Fork-Join Parallel

10 million integers, `max_depth` = 3, median of three runs:

Version	Time (sec)	Speedup
Sequential	0.5320	1.00×
Parallel (8 threads)	0.1027	5.18×

The fork-join version reaches 5.18× with 8 concurrent leaf threads, about 65% of ideal linear scaling.

5.2 Merge Sort: Depth Sweep

To map where the optimal depth sits, we swept `max_depth` from 0 (purely sequential) to 14 (16384 leaf threads), three trials per depth:

max_depth (max thread)	Parallel (s)	Speedup
0 (1)	0.5291	1.01×
1 (2)	0.2815	1.89×
2 (4)	0.1587	3.35×
3 (8)	0.0985	5.43×
4 (16)	0.0925	5.78×
5 (32)	0.0902	5.93×
6 (64)	0.0868	6.10×
7 (128)	0.0867	6.11×
8 (256)	0.0858	6.22×
10 (1024)	0.0922	5.81×
12 (4096)	0.2439	2.22×
14 (16384)	1.8520	0.28×

The data trace a classic U-shape; interpretation is in Part 6.2.

5.3 Matrix Multiplication: Three Versions
1024 × 1024 with `OMP_NUM_THREADS` = 12:

Version	Time (sec)	Speedup
Sequential (i-j-k)	5.5671	1.00×
OpenMP basic (i-j-k)	0.7800	7.14×
OpenMP optimized (i-k-j)	0.0576	96.64×

Basic OpenMP reaches 7.14× across 12 threads. The optimized version, which adds loop reordering to i-k-j on top of OpenMP, reaches 96.64×, far exceeding the thread count. The gap is analyzed in Part 6.3.

5.4 Matrix Multiplication: Loop Order Study
All six loop orderings as pure sequential C (no OpenMP), 1024 × 1024, median of two runs:

Order	Time (s)	vs i-j-k	Innermost stride pattern
i-k-j	0.181	14.1×	A fixed, B stride 1, C stride 1
k-i-j	0.209	12.2×	A fixed, B stride 1, C stride 1
i-j-k	2.555	1.00×	A stride 1, B stride N, C fixed
j-i-k	5.254	0.49×	A stride 1, B stride N, C fixed
k-j-i	7.348	0.35×	A stride N, B fixed, C stride N
j-k-i	12.76	0.20×	A stride N, B fixed, C stride N

The six orderings cluster into three groups by innermost loop index; the best-to-worst spread is more than 70×. Part 6.3 explains the grouping in terms of cache-line behaviour.

6 Discussion

6.1 Applying Amdahl's Law to Merge Sort

Solving Amdahl's Law for $N = 8$ and $S = 5.18$: $1/5.18 = 0.1931$, so $f + (1 - f)/8 = 0.1931$, which gives $(7/8)f = 0.0681$ and therefore $f \approx$ **0.078 (7.8%)**. Almost 8% of wall-clock time is inherently serial.

This serial time comes mostly from the final merge passes: the outermost merge, combining the two halves into the final sorted output, is by construction single-threaded and runs in $O(N)$ time. Thread creation and join overhead contribute the rest. For comparison, the Week 4 histogram assignment measured $f \approx 15\%$ on the same machine; merge sort's lower f reflects its higher compute-to-overhead ratio.

6.2 Oversubscription: Why MAX_DEPTH Has a Sweet Spot

The depth sweep in Part 5.2 traces a sharp U-curve. Depth 0 → 3: speedup climbs nearly linearly (1.01× → 5.43×) because each doubling of leaf threads roughly doubles parallelism up to the 8-P-core limit. Depth 3 → 8: saturation around 6.1–6.2×; adding more threads than there are physical cores yields only tiny gains. Depth 10 → 14: the curve collapses. At depth 12 (4096 threads) the program is already 3× slower than its peak, and at depth 14 (16384 threads) it is **3× slower than the single-threaded baseline** — `pthread_create/pthread_join` overhead, stack memory pressure, and scheduler contention completely obliterate the parallelism benefit.

The empirical optimum is depth 6–8, sitting just above $\log_2(\text{cores}) \approx 3.6$. That the peak 6.2× falls short of the ideal 8× (eight P-cores) is itself informative: the missing 1.8× is spent on the sequential final merges identified in Part 6.1 and on memory-bandwidth contention across the shared L2 cache. Both Amdahl's serial fraction and the memory hierarchy's bandwidth ceiling show up in the same single number.

6.3 Loop Reordering and Cache Behavior

The six orderings in Part 5.4 group by which index varies innermost:

Innermost = j (i-k-j, k-i-j, 0.18–0.21 s). Both `B[k][j]` and `C[i][j]` are stride-1 because j increments the column index in row-major storage; `A[i][k]` is loop-invariant and hoisted into a register. Best case.

Innermost = k (i-j-k, j-i-k, 2.5–5.3 s). `A[i][k]` is stride-1, but `B[k][j]` is stride-N because k advances the row index of B, so nearly every B load misses the cache. Roughly 15× to 30× slower than the best group. The spread between i-j-k (2.55 s) and j-i-k (5.25 s) reflects temporal reuse of A rows that i-j-k exploits but j-i-k loses.

Innermost = i (j-k-i, k-j-i, 7.3–12.8 s). Both `A[i][k]` and `C[i][j]` are stride-N: every read of A and every read-modify-write of C misses. About 70× slower than the best group.

Notice that i-k-j (0.18 s) versus j-k-i (12.76 s) represents a single permutation of three nested loops with identical arithmetic — **every speedup comes from memory access order alone**. On modern CPUs, memory access patterns matter more than the number of FLOPs. The ~14× gap between i-j-k and i-k-j measured here matches the ~14× gap between basic and optimized OpenMP in Part 5.3, confirming that the optimized version's entire advantage comes from loop reordering, not from anything OpenMP does differently.

6.4 Beyond Loop Reordering: Roofline and Cache Blocking

Loop reordering is a *local* fix: it removes the worst stride pathology but does not approach peak compute. A quick roofline check: 1024^3 matmul is $\approx 2.15 \times 10^9$ FLOPs, so our times give Sequential 0.39 GFLOP/s, OpenMP basic 2.75 GFLOP/s, and OpenMP optimized 37 GFLOP/s. The M2 Pro P-core peak is ~14 GFLOP/s (double-precision FMA at 3.5 GHz with 128-bit NEON), so the 8-P-core

compute ceiling is ~110 GFLOP/s for native arm64. Our 37 GFLOP/s is roughly a third of that; tuned BLAS libraries reach 70–90% of peak through register blocking, cache blocking, and hand-tuned SIMD.

The next optimization beyond loop reordering is **cache blocking** (tiling): partition each matrix into $B \times B$ sub-blocks sized so that three blocks (one from A, one from B, one from C) fit simultaneously in L1, and compute one output block at a time, keeping each reused block hot in cache until it has contributed to every output it affects. With $B \approx 64$ on the M2 P-core ($3 \times 64^2 \times 8$ bytes ≈ 96 KB, fitting in the 128 KB L1d), another 2–4× speedup is typical. Production libraries nest multiple levels of blocking (registers → L1 → L2) and write the innermost micro-kernel in hand-tuned SIMD assembly, which is why they reach near-peak performance and we do not.

6.5 Relation to Java's ForkJoinPool

Java's `ForkJoinPool` (Silberschatz 4.5.2) implements the same fork-join pattern as our Pthreads code but with two key differences. First, **managed thread pool**: a fixed set of workers is created at pool startup and reused across tasks, so fork and join operations do not incur per-call `pthread_create`/`pthread_join` overhead. Second, **work-stealing**: each worker maintains its own deque of pending tasks, and when idle it steals tasks from the bottom of other workers' queues, load-balancing irregular recursion trees automatically. Our Pthreads implementation approximates the first property through `MAX_DEPTH` but has no work stealing, so load imbalance is absorbed by the fixed fork-join structure.

6.6 Fork-Join versus OpenMP: When to Use Each

Fork-join with explicit Pthreads fits irregular, recursive parallelism (divide-and-conquer algorithms, tree and graph traversals, adaptive quadrature): the programmer controls thread creation, synchronization, and granularity explicitly. **OpenMP** fits regular data parallelism where independent loop iterations can be distributed with a single pragma (dense linear algebra, image processing, numerical grids); pooling and scheduling are handled by the runtime. Practical rule: if the parallelism naturally expresses itself as a `for` loop with independent iterations, write OpenMP; if it expresses itself as a recursive function with two sub-calls per node, write fork-join.

7 Key Takeaways

1. Amdahl's Law is the first thing to measure.
Solve for the serial fraction f before adding cores: the upper bound $1/f$ is often tighter than the core count. Merge sort in this assignment has $f \approx 7.8\%$, capping speedup near 13× regardless of how many threads are thrown at it.

2. Oversubscription is catastrophic.
The right thread count is roughly the physical core count, not a large number. In the depth sweep, 16384 threads run **3× slower** than a single thread because per-thread creation and stack costs scale linearly while the parallelism payoff saturates after ~8 threads.

3. Memory access patterns dominate FLOP count on modern CPUs.
A single loop permutation (i-j-k → i-k-j) produces a ~14× speedup without changing any arithmetic. Row-major stride-1 access in the innermost loop is the single most important matmul optimization before SIMD and blocking.

4. Choose the parallelism model by problem shape.
Regular `for` loops with independent iterations → OpenMP (one pragma, pooling automatic). Recursive divide & conquer structures → fork-join (explicit Pthreads or a work-stealing runtime).